RICE UNIVERSITY

Precision and the Anatomy of a Matrix Multiply

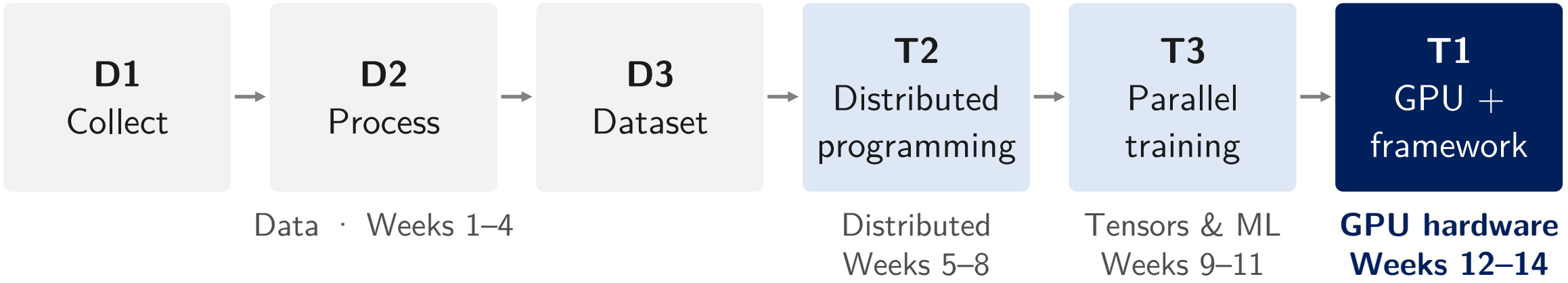
How many bits a number needs, and how a fast matrix multiply moves them

Yuke Wang

Rice University

COMP 329 · Spring 2027 · Week 14, Lecture 1

Where we are



In our pipeline

Week 13 taught us to **predict** a kernel. Today: the matrix multiply — bits per number and tiles. **Lab 3** is due after today; **T1** on Friday.

Today's plan

1

Number formats

What does a 16-bit or 8-bit number give up?

2

Mixed precision

How do we train in 16 bits and still get FP32 answers?

3

The anatomy of a matrix multiply

Why does a naive matmul leave the GPU idle, and how do tiles fix it?

Build

Formats and tiles

Work in pairs, on paper. About ten minutes.

Why do we need fewer bits and tiles?

The job: $C = A \cdot B$, 4096×4096 matrices on one H100
$\approx$ **1.4×10^{11}** FLOPs, almost all multiply-adds

One thread per C[i, j], FP32, straight from HBM: how long?

Simplest kernel, FP32 from HBM
$\approx$ **164 ms**

BF16 tensor-core peak
$\approx$ **0.14 ms**

Key idea

A gap of $\approx$ **1,200×**. Fewer **bits** per number and **reuse** of each number on chip close it.

Source: H100 datasheet (SXM5): 3.35 TB/s HBM, 989 TFLOP/s BF16 dense; derived values.

Precision and tiling, in plain words

Idea 1

Precision: bits per number.

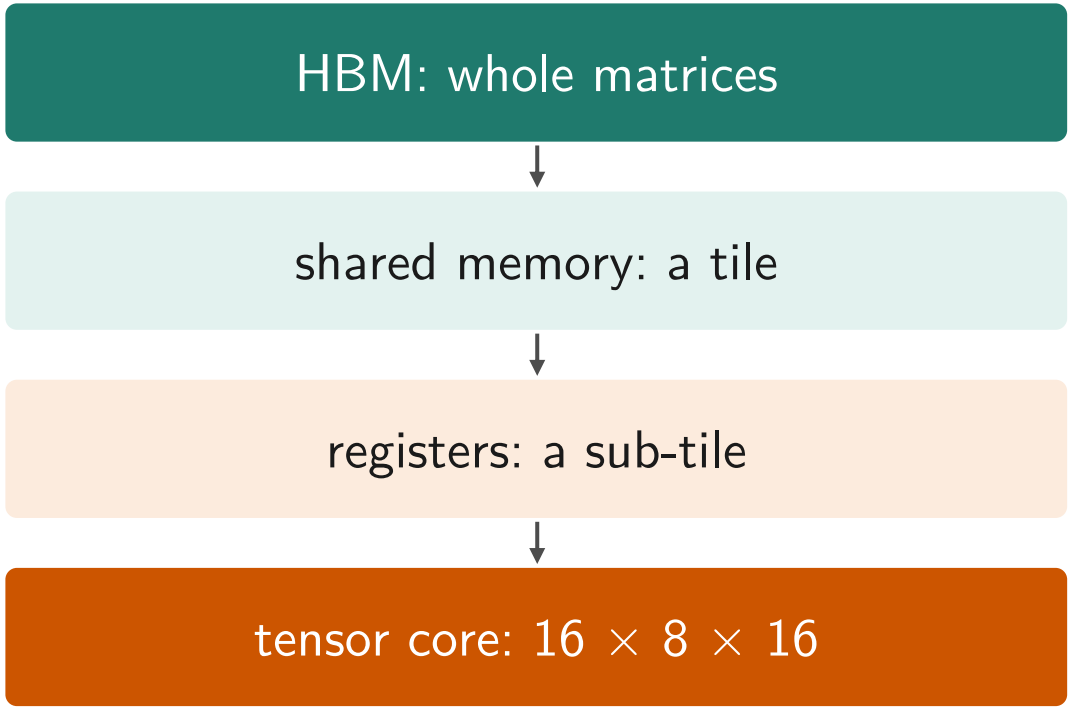
Fewer bits $\rightarrow$ fewer bytes, faster math.

Cost: less **range** or **precision**.

Idea 2

Tiling: load a block on chip **once**,

then reuse it **many** times.



Key idea

Both ideas cut **bytes moved per FLOP**, the lever the roofline (Week 13) points to.

Where they show up

- **BF16**: the usual training format; PyTorch **autocast** picks it per operation.
- **FP8**: doubles the H100 peak again; used to train large models (DeepSeek-V3).
- **Tiling**: inside cuBLAS (behind `torch.matmul`), CUTLASS, and Triton's `tl.dot`.

The price

Low precision needs a **recipe** (Part 2)

Big tiles cost **shared memory** and **registers**

In our pipeline

Why? A matrix multiply is most of a training step; bits and tiles decide whether it runs near peak. Lab 3 and T1 ask you to predict exactly this.

Source: H100 datasheet (SXM5); DeepSeek-AI, "DeepSeek-V3 Technical Report," 2024; NVIDIA cuBLAS and CUTLASS docs.

Learning goals for today

- Read a format's bits as **range** (exponent) and **precision** (mantissa): FP32, BF16, FP16, FP8.
- Explain **mixed precision**: low-precision inputs, **FP32 accumulation**, loss scaling.
- Show that a **naive** matrix multiply is **memory-bound**, and compute what **tiling** buys.
- Describe tiling through **shared memory**, **registers**, and **tensor cores**, and why tile shapes matter.

A puzzle to start

```
s = bf16(0.0)
for _ in range(4096):
    s = bf16(s + 1.0)     # round every time
print(s)
```

BF16: 16 bits per number
the usual training format

What does it print?

Watch out

It prints **256**. Above 256, BF16 numbers are **2 apart**, so 256 + 1 rounds back to 256.
Long sums need **more bits** than the inputs.

Part 1

Number formats

What does a 16-bit or 8-bit number give up?

1 · Number formats

2 · Mixed precision

3 · The anatomy of a matrix multiply

Build · Formats and tiles

Anatomy of a floating-point number

A number is stored as **sign** $\times$ **(1 +** **fraction)** $\times$ **a power of 2**.

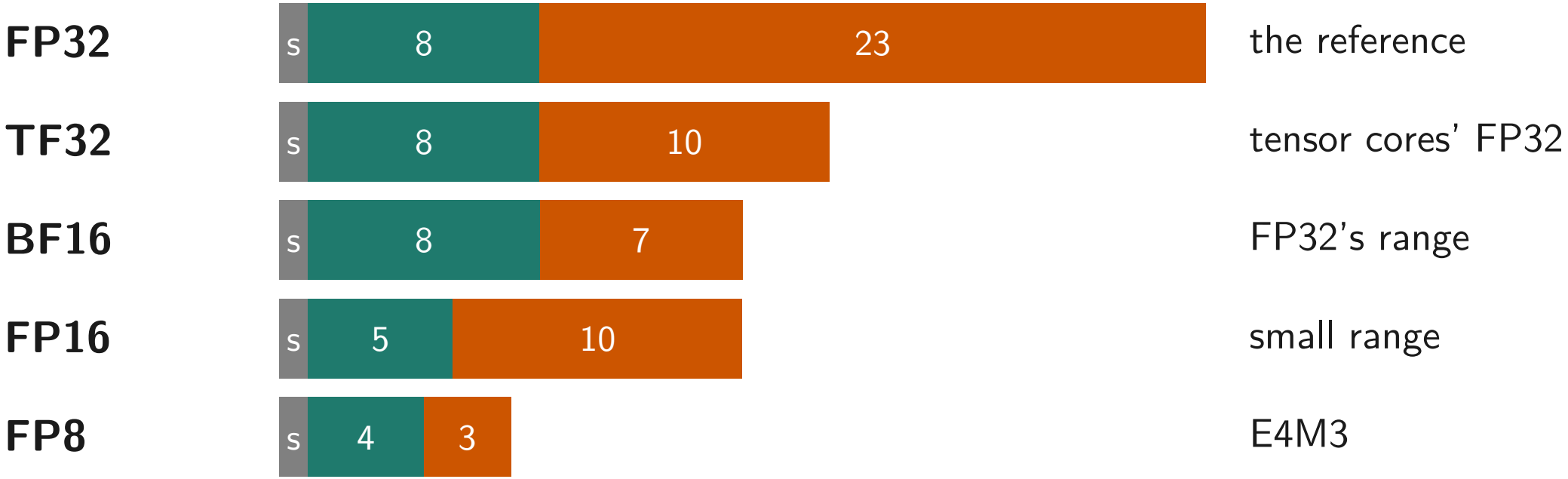


- **Exponent** bits $\rightarrow$ **range**: how large or small a number can be.
- **Mantissa** bits $\rightarrow$ **precision**: the gap after 1 is $\varepsilon = 2^{-23} \approx 1.2 \times 10^{-7}$.

Source: IEEE 754-2019 (FP32, FP16); NVIDIA A100 and H100 whitepapers (BF16, TF32); Micikevicius et al., 2022 (FP8).

Five formats, bit by bit

grey: sign · teal: exponent bits · orange: mantissa bits · one cell = one bit



Key idea

BF16 keeps FP32's **range** and gives up **precision**; FP16 does the opposite.

Source: IEEE 754-2019 (FP32, FP16); NVIDIA A100 and H100 whitepapers (BF16, TF32); Micikevicius et al., 2022 (FP8).

Range versus precision

Format	Bits	Largest value	ε (gap after 1)
FP32	32	$\approx 3.4 \times 10^{38}$	1.2×10^{-7}
BF16	16	$\approx 3.4 \times 10^{38}$	7.8×10^{-3}
FP16	16	65,504	9.8×10^{-4}
FP8 (E4M3)	8	448	0.125

Key idea

Exponent bits buy range; mantissa bits buy precision. With 16 bits you choose one; with 8 bits you need help (Part 2).

Source: IEEE 754-2019 (FP32, FP16); NVIDIA A100 and H100 whitepapers (BF16, TF32); Micikevicius et al., 2022 (FP8).

Tensor cores and precision

Format	Bits	H100 peak (dense)	Typical use
FP32 (ordinary lanes)	32	$\approx$ 67 TFLOP/s	Reference math
TF32 (tensor cores)	19	$\approx$ 495 TFLOP/s	Faster FP32 matmuls
BF16 / FP16 (tensor cores)	16	$\approx$ 989 TFLOP/s	Training default
FP8 (tensor cores)	8	$\approx$ 1,979 TFLOP/s	Large-model training

Watch out

These peaks apply **only to matrix products**. Elementwise work (tanh, softmax, adds) runs on ordinary lanes and is usually memory-bound.

Source: H100 datasheet (SXM5); tensor-core values are the datasheet's sparse peaks divided by 2.

Two ways low precision fails

Underflow (range)
gradient g $= 10^{-8}$
FP16: rounds to **0**
BF16, FP32: kept

Swamping (precision)
w + update $= 1 + 10^{-4}$
BF16, FP16: $=$ **1** (update lost)
FP32: kept

Watch out

Small numbers **vanish** (too little range) or small changes **vanish** (too little precision). Part 2 fixes both.

Source: IEEE 754-2019; checked in Python (NumPy float16, BF16 rounding emulated).

Check yourself

Two minutes with a neighbour.

- An activation reaches 70,000. Which of FP16 and BF16 can store it? Why?
- What is the next FP8 (E4M3) number after 1? What does 1.1 become?
- A weight $w = 1$ is kept in BF16 and updated by 10^{-3}. What happens?

Part 2

Mixed precision

How do we train in 16 bits and still get FP32 answers?

1 · Number formats

2 · Mixed precision

3 · The anatomy of a matrix multiply

Build · Formats and tiles

Multiply in BF16, add up in FP32

BF16 inputs
a[k], b[k]
2 bytes each

→

product a[k] · b[k]
exact

→

FP32 accumulator
s += a[k] · b[k]

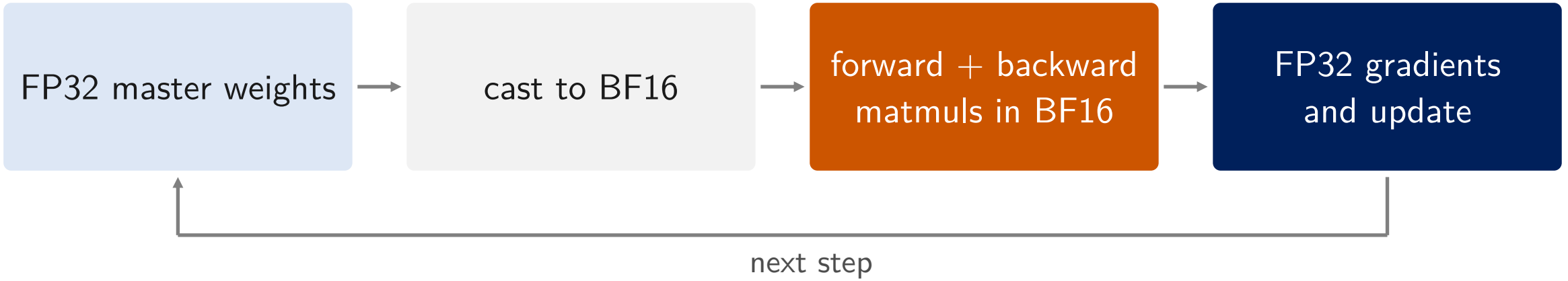
Sum of 4,096 ones	Result
BF16 accumulator	**256**
FP32 accumulator	4,096

Key idea

Round the **inputs**, never the **running sum**.

Source: NVIDIA PTX ISA, warp-level matrix multiply (mma.m16n8k16 for BF16, FP32 accumulator); H100 whitepaper.

Mixed precision: the recipe



Key idea

Matrix multiplies in **BF16** for speed; sums, softmax, the loss, and the update in **FP32** for accuracy.

Source: Micikevicius et al., "Mixed Precision Training," ICLR 2018; PyTorch AMP docs.

FP16 needs loss scaling; BF16 does not

$$\text{scaled loss: } \mathcal{S} \cdot L, \quad \text{for example } \mathcal{S} = 2^{16}$$

$$\text{chain rule: } \partial(\mathcal{S} \cdot L) \,/\, \partial w = \mathcal{S} \cdot \partial L \,/\, \partial w$$

$$10^{-8} \times 2^{16} \approx 6.6 \times 10^{-4} \quad \text{fits in FP16}$$

$$\text{before the update: divide by } \mathcal{S} \text{ (in FP32)}$$

Check

BF16 has FP32's exponent range: gradients do not underflow, so it needs **no scaling**.

Source: PyTorch docs, "Automatic Mixed Precision" (torch.amp); Micikevicius et al., "Mixed Precision Training," ICLR 2018.

FP8: a scale factor per tensor

$$\text{scale:} \quad s = 448 \;/\; \text{amax}(x)$$

$$\text{amax} = 7 \quad \Rightarrow \quad s = 64$$

$$\text{store} \;\; x \cdot s \;\; \text{in FP8;} \;\; \text{later divide by} \;\; s$$

Why

E4M3 holds about

0.002 … 448

Most tensors do not fit without a scale.

Key idea

FP8 = loss scaling **per tensor**: one FP32 scale each, so every tensor uses FP8's narrow range well.

Source: Micikevicius et al., "FP8 Formats for Deep Learning," 2022; NVIDIA Transformer Engine documentation.

Mixed precision in PyTorch

```
model = MLP().cuda()              # FP32 weights
for x, y in loader:
    with torch.autocast("cuda",
                        dtype=torch.bfloat16):
        loss = F.cross_entropy(model(x), y)
    loss.backward()                  # FP32 .grad
    opt.step()                       # FP32 update
    opt.zero_grad()
```

- Wrap the **forward and loss** only.
- Parameters and gradients stay **FP32**.

Key idea

Autocast picks the format **per operation**: matrix multiplies go low, sums and the loss stay FP32.

Source: PyTorch docs, "Automatic Mixed Precision" (torch.amp); Micikevicius et al., "Mixed Precision Training," ICLR 2018.

Check yourself

Two minutes with a neighbour.

- Why does a dot product of 4,096 BF16 products need an FP32 accumulator?
- Which format needs loss scaling, FP16 or BF16? Why?
- A tensor has amax $= 0.5$. What FP8 (E4M3) scale factor do you use?

Part 3

The anatomy of a matrix multiply

Why does a naive matmul leave the GPU idle, and how do tiles fix it?

1 · Number formats | 2 · Mixed precision | **3** · The anatomy of a matrix multiply | **Build** · Formats and tiles

Recall: FLOPs and the roofline

$$\mathcal{F} = 2 \cdot \mathcal{M} \cdot \mathcal{N} \cdot \mathcal{K} \qquad \text{(Week 11)}$$

$$\mathcal{T} \geq \max(\ \mathcal{F} \,/\, \pi\ ,\ \ \mathcal{Q} \,/\, \beta\ ), \qquad \mathcal{I} = \mathcal{F} \,/\, \mathcal{Q} \qquad \text{(Week 13)}$$

$$\text{ridge}\ \ \pi \,/\, \beta\text{:}\quad \text{FP32} \approx 20, \quad \text{BF16} \approx 295\ \ \text{FLOP/byte}$$

Key idea

A kernel is compute-bound only if it does more FLOPs per byte than the ridge. **What does a matmul kernel achieve?**

Source: H100 datasheet (SXM5); Williams, Waterman & Patterson, CACM 2009.

A naive matmul is memory-bound

```
i, j = my_output()          # one C[i, j]
acc = 0.0
for k in range(K):
    acc += A[i, k] * B[k, j]    # 2 loads
C[i, j] = acc
```

$$\text{reads} \approx 2\text{MNK values}$$

$$I = 2\text{MNK} \;/\; (2\text{MNK} \cdot b) = 1 \;/\; b$$

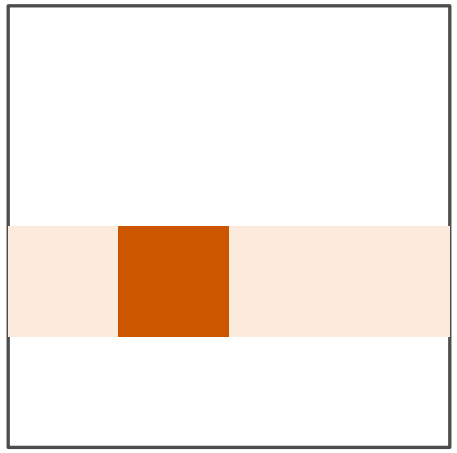
$$\text{FP32:}\quad I = 1/4 \;\ll\; 20$$

Watch out

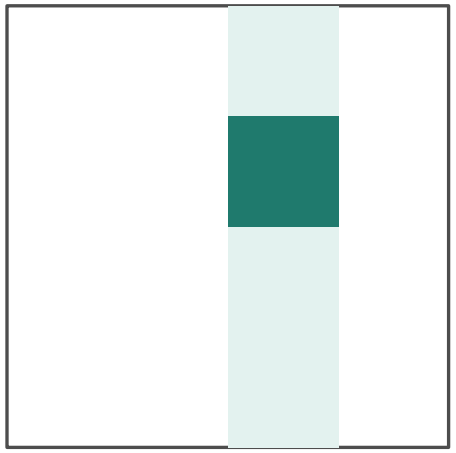
Every value is fetched from HBM **K times**. In FP32 the kernel reaches $\approx$ **0.84 TFLOP/s**: about **1%** of the FP32 peak.

Source: H100 datasheet (SXM5), 3.35 TB/s; derived values.

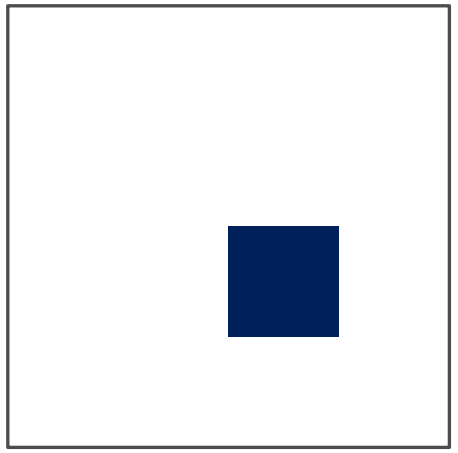
Tiling: reuse data on chip



A (M × K)



B (K × N)



C (M × N)

Naive: each output re-reads a whole row of A and column of B from HBM.

Tiled: load one T × T tile of A and of B into shared memory; use each value **T times**.

Key idea

HBM traffic falls by a factor of ≈ **T**: tiles move once, then are reused on chip.

Tiling, quantified

$$\text{naive:}\quad \text{reads} \approx 2\text{MNK values} \quad \Rightarrow \quad I = 1 \,/\, b$$

$$\text{tiled:}\quad \text{reads} \approx 2\text{MNK} \,/\, T\,\text{values} \quad \Rightarrow \quad I \approx T \,/\, b$$

$$\text{FP32},\ T = 64\text{:}\ \ I = 16 \qquad\qquad \text{BF16},\ T = 128\text{:}\ \ I = 64$$

Watch out

Lower precision raises the ridge to ≈ 295: in BF16, tiling is **necessary**, and one level of it is **not enough**.

Source: Derived; ridge points from the H100 datasheet (SXM5).

How big can a tile be?

$$\text{shared memory per } k\text{-step:} \quad (\mathrm{BM} \cdot \mathrm{BK} + \mathrm{BK} \cdot \mathrm{BN}) \cdot b$$

$$\mathrm{BM} = \mathrm{BN} = 128, \quad \mathrm{BK} = 64, \quad \mathrm{BF16}: \quad 32 \text{ KB}$$

$$\text{double-buffered:} \quad 64 \text{ KB} \quad \text{of } 228 \text{ KB per SM}$$

Memory

Tiles are capped by **shared memory and registers**. The **L2 cache** serves the tiles that neighbouring blocks re-read, so HBM sees less.

Source: NVIDIA H100 Tensor Core GPU datasheet (SXM5); NVIDIA H100 Tensor Core GPU Architecture whitepaper (2022).

Register tiling: a second level



per k: $2r$ values in, r^2 multiply-adds

$$I = 2r^2 \,/\, (2r \cdot 4) = r \,/\, 4$$

$$r = 8: \quad 2 \times 33 \text{ TB/s} \approx 67 \text{ TFLOP/s}$$

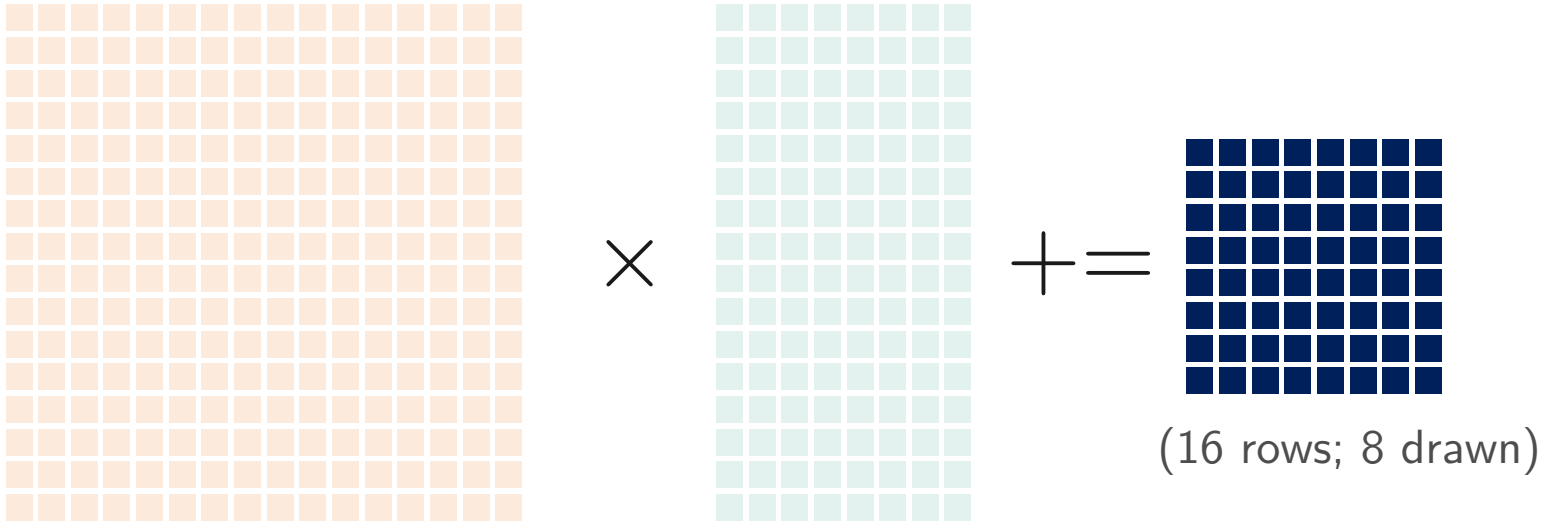
Key idea

Each level of memory needs its own reuse: **shared-memory tiles** for HBM, **register tiles** for shared memory.

Source: H100 whitepaper (128 bytes per cycle per SM, 132 SMs); derived, as in Week 13.

Tensor cores multiply tiles

A: 16 × 16 (BF16) **B: 16 × 8 (BF16)** **C: 16 × 8 (FP32)**



(16 rows; 8 drawn)

mma 16 × 8 × 16

One instruction, one warp:

4,096 FLOPs

inputs BF16, sum **FP32**

Key idea

Tensor cores consume **small tiles from registers**. Shared-memory and register tiling exist to **feed** them.

Source: NVIDIA PTX ISA, warp-level matrix multiply (mma.m16n8k16 for BF16, FP32 accumulator); H100 whitepaper.

Tile shapes: pad to a multiple

N = 322 columns, computed in 64-wide tiles



384 computed; 62 are padding

$$\text{64-wide tiles:} \quad \lceil 322 \,/\, 64 \rceil \cdot 64 = 384 \quad (16\% \text{ wasted})$$

$$\text{16-wide tiles:} \quad \lceil 322 \,/\, 16 \rceil \cdot 16 = 336 \quad (4\% \text{ wasted})$$

Watch out

Choose dimensions that are **multiples of the tile** (16, 64, 128): otherwise you pay for padding.

Source: NVIDIA Deep Learning Performance Guide, "Matrix Multiplication Background" (tile and wave quantization).

Three levels of tiles, together

HBM
whole A, B, C

Shared memory · per block
128×64 slices of A and B

Registers · per warp
64×64 FP32 accumulators

Tensor core · per instruction
$16 \times 8 \times 16$, BF16 in, FP32 sum

smaller tiles,
faster memory,
more reuse

Key idea

Each level **reuses** what the level above delivered. Next lecture, Triton writes the lower levels for you.

Check yourself

Two minutes with a neighbour.

- A naive BF16 matmul: what intensity? Memory- or compute-bound on an H100?
- What intensity do $T = 32$ tiles give in FP32? $T = 128$ tiles in BF16?
- Why make a layer's width 384 rather than 322?

IN-CLASS BUILD

Formats and tiles

Work in pairs, on paper. About ten minutes.

1 · Number formats

2 · Mixed precision

3 · The anatomy of a matrix multiply

Build · Formats and tiles

Exercise A: pick a format

Part of the training step	Format?
Master weights and the update $w \leftarrow w - \eta \cdot g$	
Inputs of the forward and backward matmuls	
Each matmul's running sum (accumulator)	
Softmax and the cross-entropy loss	

STRETCH

The GPU has no BF16, only FP16. What must you add, and why?

Exercise B: count a tiled matmul

$$\mathbf{C = A \cdot B},\ \ M = N = K = 4096,\ \ \text{BF16}$$
$$\text{tiles: } BM = BN = 128,\ \ BK = 64$$

- **1.** FLOPs F.
- **2.** Shared memory per block for one k-step of A and B.
- **3.** Bytes read into the SMs, and the intensity I.
- **4.** Memory- or compute-bound against the BF16 ridge? What T would reach it?

USE

$\beta = 3.35$ TB/s, $\pi = 989$ TFLOP/s, ridge ≈ 295. Stretch: question 4 in FP32 (ridge ≈ 20).

Source: NVIDIA H100 Tensor Core GPU datasheet (SXM5); NVIDIA H100 Tensor Core GPU Architecture whitepaper (2022).

Solutions A: formats

Part of the training step	Format
Master weights and the update	**FP32**: no swamping
Matmul inputs	**BF16**: tensor cores
Matmul accumulator	**FP32**: the puzzle
Softmax and loss	**FP32**: exp, log, sums

Check

FP16 only: **loss scaling**, because its small range makes gradients underflow.

Solutions B: a tiled matmul

Question	Answer
1. F	$2 \cdot 4096^3 \approx 1.37 \times 10^{11}$ FLOPs
2. Shared memory	$(128 \cdot 64 + 64 \cdot 128) \times 2$ B $= 32$ KB (64 KB double-buffered)
3. Bytes, I	$2MNK \cdot 2 / 128 \approx 2.1$ GB; $I = T / b = 64$
4. Bound?	$64 < 295$: memory-bound from HBM; needs $T \approx 590$

Memory

$T \approx 590$ would need a **1.4 MB** accumulator: impossible. The **L2 cache** closes the rest of the gap.

Source: NVIDIA H100 Tensor Core GPU datasheet (SXM5); NVIDIA H100 Tensor Core GPU Architecture whitepaper (2022).

Key takeaways

- **Exponent** bits buy range, **mantissa** bits buy precision: BF16 keeps range, FP16 keeps precision.
- Mixed precision: **BF16 inputs**, **FP32 accumulation**, FP32 weights and update; FP16 adds **loss scaling**, FP8 a **scale per tensor**.
- A **naive** matmul has $\mathrm{I} = 1/\mathrm{b}$: **memory-bound**.
- **Tiling** gives $\mathrm{I} \approx \mathrm{T}/\mathrm{b}$; **register tiles** and **tensor cores** repeat the idea one level down.
- Pad dimensions to **multiples of the tile**.

Next time, and readings

When	What
After today	Lab 3 due
Lecture 2 (this week)	Triton, and one training step end to end
Friday, April 23	T1 due

READ BEFORE LECTURE 2

Micikevicius et al., "Mixed Precision Training," ICLR 2018.

NVIDIA, "Matrix Multiplication Background" (Deep Learning Performance Guide).

Tillet, Kung & Cox, "Triton," MAPL 2019 (for Lecture 2).